SIMATS ENGINEERING

Co3-ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks **COURSE CODE: CSA07**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS:4 **Total Marks:40**

Code of Conduct:

I _T.Ganesh(192425246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
T.Ganesh
Signature of the student:

Criteria	Understanding the case 2.5	**Application of Concepts** 2.5	**Analysis** 2	**Solution Design** 2	**Clarity** 1	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

Question 1: Cloud-based Video Conferencing Issues

Scenario: A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

1.1 Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer

Answer: The problem is primarily at the **application layer** (buffering and adaptation strategy), with an unsuitable transport protocol choice as a strong contributing factor.

Key observations:

- Symptoms: 8% packet loss, 180 ms jitter, fluctuating bandwidth, voice distortion, frozen frames, delayed communication.
- These are classic real-time multimedia problems.

- Transport protocol (TCP vs UDP) contributes, but the decisive factor is how the application reacts to loss, jitter, and bandwidth variation.

1.2 Justify your answer based on the communication requirements of real-time multimedia applications

Real-time video/voice conferencing has strict requirements:

- Low latency (ideally < 150–200 ms one-way).
- Tolerance of some packet loss (better to drop a frame/packet than wait for retransmission).
- Continuous playout (smooth audio/video even under variable conditions).

Why TCP is problematic: TCP provides reliable, ordered delivery with congestion control and retransmissions. Under 8% loss and high jitter this causes head-of-line blocking, retransmission delays that push latency far beyond interactive limits, and congestion-window collapse that further reduces throughput.

Why UDP + application control is preferred: UDP is connectionless and does not retransmit by itself. It lets the application decide what to do with loss and jitter. When the application layer has poor buffering, adaptive bitrate, or jitter-buffer logic, the exact

symptoms described (distortion, freezes, delay) appear. Therefore the transport protocol is a contributing factor (especially if TCP is used), but the dominant issue is the application-layer strategy for real-time media.

1.3 Recommend suitable protocol-level or application-level improvements

Protocol / Transport Level

- Prefer UDP + RTP/RTCP (or WebRTC's SRTP) over pure TCP.
- Use QUIC (UDP-based) if supported; it provides multiplexing, congestion control, and 0-RTT without TCP's head-of-line blocking.
- Enable Forward Error Correction (FEC) or selective retransmission only for key frames / audio.

Application Level

- Adaptive jitter buffer that dynamically adjusts playout delay according to measured jitter (target ~20–50 ms under normal conditions; expand under 180 ms spikes).
- Adaptive bitrate / Scalable Video Coding (SVC) so resolution and frame-rate drop gracefully when bandwidth fluctuates.
- Packet-loss concealment (PLC) and error-resilient video coding.

- Prioritize audio over video (audio is more sensitive to delay and loss).
- QoS marking (DSCP EF for voice, AF for video) and network-side traffic shaping or prioritization where possible.
- Monitor RTCP reports and react in real time (switch codecs, reduce layers, etc.).

Question 2: High DNS Lookup Time in a Multinational Company

Scenario: A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

2.1 Analyze the reasons behind the high DNS resolution time

- Most or all queries reach a distant or overloaded primary DNS server (geographic distance causes high RTT).
- Lack of local caching / recursive resolvers near users.
- Long or poorly tuned TTLs leading to frequent full recursive lookups.

- No anycast or geo-distributed authoritative servers, so every lookup travels inter-continentally.
- Possible recursive resolution delays (many referrals, slow root/TLD servers, or rate-limiting).
- Clients not performing DNS prefetching or concurrent lookups.

2.2 Propose a suitable DNS architecture (Anycast DNS, DNS pre-fetching, TTL optimization) to reduce lookup time below 50 ms

- **Anycast DNS:** Deploy multiple authoritative name-server instances (or anycast IPs for recursive resolvers) in each major continent/region. BGP anycast automatically routes the client to the nearest instance (typical RTT < 20–30 ms).
- **Local recursive resolvers + caching:** Place caching recursive resolvers close to employee networks / branch offices (or use public anycast resolvers such as 1.1.1.1 / 8.8.8.8). Aggressive positive and negative caching.
- **TTL optimization:** Short TTL (30–60 s) for records that change frequently; longer TTL (5–15 min or more) for stable records. Combine with DNS pre-fetching / pre-resolution at the client or CDN edge so the lookup is already warm.

- **Secondary / multi-master authoritative servers:** Zone transfers or real-time replication so any single site failure is transparent.
- **Optional enhancements:** DNS over HTTPS/TLS for privacy; client-side stub-resolver caching and concurrent A/AAAA queries.

Resulting lookup path: Client → nearest anycast recursive resolver (cache hit or very short recursive query) → nearest anycast authoritative → answer returned in < 50 ms in the common case.

2.3 Evaluate advantages and limitations of the proposed architecture

Advantages

- **Performance:** Geographic proximity + caching drives latency well below 50 ms.
- **Scalability:** Anycast + multiple instances absorb global query load; each site scales independently.
- **Availability:** Failure of one site simply removes that anycast route; traffic automatically shifts to the next nearest healthy site (continuous service).

Limitations

- Operational complexity: managing anycast BGP, zone consistency, and monitoring many sites.
- Cache consistency / TTL trade-off: very short TTLs increase query volume; very long TTLs slow fail-over after a record change.
- Cold-cache / first-query latency can still be higher until the cache warms.
- Cost of global infrastructure and anycast routing expertise.

Question 3: Centralized DNS Architecture at a Cloud Service Provider

Scenario: A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

3.1 Examine the impact of centralized DNS architecture on application performance

A single primary DNS server becomes a:

- Latency bottleneck – every query travels a long distance.
- Single point of failure – outage or overload makes the whole service unreachable or extremely slow.
- Scalability bottleneck – cannot handle promotional traffic spikes.

Result: Application homepage / API calls suffer multi-hundred-millisecond delays, poor user experience, and cascading load on the application tier when retries occur.

3.2 Design a distributed DNS solution that minimizes lookup latency while maintaining high availability

- Deploy anycast authoritative DNS servers in multiple PoPs / continents (same anycast IP announced from many locations).
- Front them with geographically distributed recursive resolvers / caches (or rely on well-known anycast public resolvers).
- Use multi-master or primary + many secondaries with automatic zone transfer / DNS NOTIFY.
- Health-check the servers; withdraw the anycast route from unhealthy sites.
- Clients and CDNs perform DNS prefetching; application can also use connection coalescing / HTTP/3.

- Optional: geo-DNS / latency-based routing for further optimization of the returned A/AAAA records.

3.3 Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance

- **TTL:** Short TTLs improve freshness and faster fail-over after a server change, but raise query rate. Long TTLs reduce load and improve cache-hit ratio (lower latency) but delay recovery from failures. Optimal strategy is tiered TTLs + monitoring.
- **Anycast routing:** Clients automatically reach the nearest healthy server → lower RTT and automatic fail-over when a site withdraws its route. Greatly improves both latency and availability.
- **DNS caching** (at recursive resolvers, OS, browser, CDN): Converts most subsequent lookups into sub-millisecond local operations. Combined with anycast it keeps the majority of queries under 50 ms even under global load. Negative caching also prevents repeated queries for non-existent names.

Together, TTL tuning, Anycast, and caching deliver low latency, high query throughput, and strong fault tolerance.

Question 4: High-Frequency Stock Trading Platform – TCP Latency Issues

Scenario: A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

4.1 Analyze three possible TCP-related factors responsible for the latency increase

Considering flow control, Nagle's algorithm, and SYN queue limitations, the three factors are:

- **Flow control / receive-window and congestion-window dynamics**
- **Nagle's algorithm** (and its interaction with delayed ACK)
- **SYN queue / listen backlog limitations** (and related connection-establishment delays)

4.2 Explain how each factor affects transaction processing in high-frequency systems

a. Flow / Congestion Control

Under load the receiver window or congestion window shrinks (or slow-start restarts after loss). Data (order messages) is held back, increasing RTT and order-ack latency from 1 ms to tens of ms. Retransmissions further amplify the delay.

b. Nagle's Algorithm

Small order messages (< MSS) are buffered until a full segment can be sent or an ACK arrives. Combined with the delayed-ACK timer (typically 40–200 ms) this can add tens of milliseconds of artificial delay — exactly the observed jump to 40 ms.

c. SYN Queue / Backlog Limits

When thousands of new connections (or rapid reconnects) arrive, the SYN queue or accept queue fills. New SYNs are dropped or delayed, TCP handshake times increase, and established connections may also suffer because the server is busy. This directly raises connection-establishment and subsequent transaction latency.

4.3 Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability

- **Disable Nagle's algorithm** (TCP_NODELAY) on the trading sockets so small order messages are sent immediately.

- **Increase listen backlog / SYN queue size** (somaxconn, tcp_max_syn_backlog) and enable SYN cookies if needed to handle connection bursts without dropping SYNs.
- **Tune congestion control:** Use a low-latency algorithm (e.g., BBR or CUBIC with appropriate parameters); consider larger initial windows if the network is known to be clean.
- **Reduce or disable delayed ACK** on the critical path if possible, or ensure the application sends data in both directions promptly.
- **Keep connections persistent** (connection pooling / long-lived TCP) to avoid repeated handshakes.
- **Optional advanced options:** Enable TCP Fast Open or use a lighter transport (UDP-based reliable protocol) for the most latency-sensitive order path, while retaining TCP for less critical traffic.
- **Monitoring & QoS:** Monitor retransmissions, RTT, and queue lengths; apply QoS so trading traffic is prioritized.

End of Solutions

These solutions address the root causes identified in each case study and provide practical, industry-aligned recommendations for improving performance, availability, and user experience in real-

time multimedia, global DNS, and high-frequency trading environments.